Wireshark for Cyber Threat Monitoring

Hands-On Lab Worksheet

Day 1 afternoon lab, 13:00 to 17:00. Baselines and anomalies: from reading packets to spotting what does not belong.

Name	Ella Faith Sigue	**Partner**	N/A
Date	Oct 20, 2026	**Operating system**	Windows 10
Wireshark version	Version 4.6.9 (v4.6.9-0-g2d548b197c75).	**Group (Part 5)**	Solo

This afternoon at a glance

Time	Activity	Min	Capture file
13:00	Setup and tool check	15	none
13:15	Part 1: Capture your own baseline (live)	40	my_baseline_fallback.pcapng (backup)
13:55	Part 2: Follow the handshake	50	web_browse.pcapng
14:45	Break	15	
15:00	Part 3: Filter scavenger hunt	40	hunt1.pcapng
15:40	Part 4: Baseline dossier	40	office_day.pcapng
16:20	Part 5: Odd one out (team challenge)	35	tuesday.pcapng
16:55	Submit and self-check	5	

Ground rules

Authorized captures only. Capture only your own computer, on the lab network, when the instructor says so. Sniffing traffic you are not authorized to see can be illegal.

The lab files are safe and synthetic. Every IP address, name, and password in them is fictional.

Treat captures as sensitive. Real captures contain passwords and personal data. Do not share them outside this class.

Save often. Put your initials in every file name you save.

How to work

- Work in pairs. One person drives the mouse; **swap at every Part**.
- Type filters exactly as shown. The filter bar turns **green** when the syntax is valid, **red** when it is not.
- The number of matching packets is in the status bar at the bottom right: **Displayed: N**.
- Stuck for more than five minutes? Ask your partner, then the next pair, then the instructor. Appendix B lists common problems.

Setup and tool check

13:00 to 13:15, 15 minutes

1. Copy the **Day1_Lab** folder from the shared drive to your desktop. Confirm it contains the five `.pcapng` files listed in the schedule above.
2. Open Wireshark, then **Help, About Wireshark**. Write the version at the top of this worksheet. You need version **4.0 or newer**.
3. Create your working profile: **Edit, Configuration Profiles**, click the **+** button, name it `Threat Hunting`, click OK. The status bar (bottom right) should now say *Profile: Threat Hunting*.
4. Turn off name resolution so Wireshark shows raw addresses: **View, Name Resolution**, and make sure all three *Resolve...* items are **unticked**.
5. Open **View, Time Display Format** and pick **Seconds Since Beginning of Capture**. (In Part 5 you will change this to UTC.)
6. Test that Wireshark works: **File, Open**, choose `web_browse.pcapng`. The status bar should show **Packets: 18**. Close the file (**File, Close**).

☐ Version is 4.0 or newer

☐ Profile `Threat Hunting` is active

☐ Name resolution is off

☐ I have read the ground rules

Part 1: Capture your own baseline

13:15 to 13:55, 40 minutes, individual, live capture

Goal: see what "normal" looks like on your own computer, and learn the difference between a capture filter and a display filter.

Steps

1. Find your **default gateway** (your router). Windows: open Command Prompt and run `ipconfig`, then read *Default Gateway*. macOS: `route -n get default | grep gateway`. Linux: `ip route | grep default`. Write it here: _192.168.1.1 or 192.168.1.47_
2. In Wireshark, click the interface you are actually using (Wi-Fi or Ethernet, the one with the moving line graph). Press `Ctrl+K` to open Capture Options and **untick "Enable promiscuous mode on all interfaces"**, so you capture only your own traffic.
3. Press `Ctrl+E` to start the capture.
4. Generate traffic. (a) Ping the gateway four times. Windows: `ping -n 4 <gateway>`. macOS or Linux: `ping -c 4 <gateway>`. (b) Run `nslookup example.com`. (c) Open one web page in your browser.
5. Wait about 10 seconds, then press `Ctrl+E` to stop. Save with **File, Save As** as `Part1_yourinitials.pcapng`.
6. Answer questions 1.1 to 1.5 using your capture. Type each filter into the display filter bar and press Enter.

7. **Capture filter experiment:** type `port 53` into the *capture filter* box (on the Wireshark start screen, or in Capture Options on the Input tab). Start a new capture, run `nslookup example.com` twice, browse a page, stop, and answer 1.6.

> **If capturing is blocked on your computer**
> Some computers do not allow packet capture (missing driver or permission). Tell the instructor, then open **my_baseline_fallback.pcapng** and answer 1.1 to 1.5 from it. Skip 1.6.

#	Question	Your answer
1.1	How many packets did you capture? (Status bar, *Packets:*)	16713
1.2	List four different protocols shown in the Protocol column.	UDP, ARP, TCP, DNS
1.3	Apply `icmp.type == 8`. How many echo requests are shown? Does it match the number of pings you sent?	I observed four ICMP echo requests, matching the four pings I sent to the default gateway.
1.4	Apply `dns`. What is the IP address of the DNS server your computer asked (the destination of a query)? Did a response come back?	I sent a DNS query to 192.168.1.1. A response did come back.
1.5	Name two protocols in your capture that you did not cause on purpose. What do you think each is for?	Two protocols in my capture that I did not cause on purpose are ARP and UDP. ARP resolves IP addresses to MAC addresses so devices can communicate on the local network. UDP is a transport protocol that allows applications to send data quickly without establishing a connection first. These protocols can operate automatically in the background during normal network activity.
1.6	In the `port 53` capture: which protocols appear, and what disappeared compared with your first capture? Explain in one sentence the difference between a capture filter and a display filter.	Only DNS packets appear in my port 53 capture, while ARP, TCP, and other unrelated traffic disappeared. This is because the capture filter port 53 records only packets using port 53, which is commonly used for DNS traffic. A capture filter determines which packets are recorded, while a display filter only determines which captured packets are displayed.

Part 2: Follow the handshake

13:55 to 14:45, 50 minutes, pairs, file: web_browse.pcapng

Goal: trace one web visit from the DNS lookup to the TCP handshake to the HTTP request, and draw it as a ladder diagram.

The capture shows a workstation (**10.0.0.22**) visiting the lab intranet server (**10.0.0.5**). Open `web_browse.pcapng` (18 packets).

Steps

1. Apply `dns`. Fill in table 2A.
2. Apply `tcp.stream eq 0`. This shows one complete TCP conversation. The status bar tells you how many packets: write it in table 2A.
3. Find the three handshake packets (look at the Info column for **[SYN]**, **[SYN, ACK]**, **[ACK]**). Write their frame numbers in 2A.
4. Click the first SYN packet. In the middle pane expand **Transmission Control Protocol, Options**. Record the **Maximum segment size** and the **Window scale multiplier** for the SYN and for the SYN/ACK.
5. Work out the time between the SYN and the SYN/ACK from the Time column (subtract). This is a rough round-trip time.
6. Right-click any packet in the stream, choose **Follow, TCP Stream** (`Ctrl+Alt+Shift+T`). Red text is the client, blue text the server. Fill in 2B, then close the window.
7. Clear the filter (click the X). Fill in the **ladder diagram** in 2C using the Time column.
8. **Bonus:** find the packet with the **RST** flag. Fill in 2D.

2A. DNS and handshake

Question	Your answer
Frame number of the DNS query, and the name being asked for	Frame 1; intranet.lab.test
Frame number of the DNS response, and the IP address it returned	Frame 2; 10.0.0.5
Number of packets shown by `tcp.stream eq 0`	14 packets (Frames 3–16)
Frame numbers: SYN / SYN-ACK / ACK	Frame 3 / Frame 4 / Frame 5
SYN packet: MSS value and window scale multiplier	MSS: 1460 bytes; multiplier: 256
SYN/ACK packet: MSS value and window scale multiplier	MSS: 1460 bytes; multiplier: 12
Time from SYN to SYN/ACK (seconds)	0.001 seconds (1 millisecond)

2B. What the Follow TCP Stream window shows

Question	Your answer
How many HTTP requests did the client send on this one connection?	The client sent 2 HTTP requests: GET / HTTP/1.1 and GET /style.css HTTP/1.1.
The first request line, and the value of the Host header	First request line: GET / HTTP/1.1. Host header: intranet.lab.test.
The User-Agent (which browser?)	Mozilla/5.0 (Windows NT 10.0; Win64; x64) Firefox/128.0. The browser is Mozilla Firefox 128.0 running on Windows 10 or a compatible Windows environment.
The status line the server answered with, and the type of each file returned	The server returned HTTP/1.1 200 OK for both requests. The first response was text/html (the HTML webpage), and the second was text/css (the CSS stylesheet).

2C. Ladder diagram

Fill in one row per event, in time order. For the direction column write **Client to Server** or **Server to Client**.

Frame	Time (s)	Direction	Flags / what it does
3	0.005499	Client to Server	Handshake step 1: [SYN] - Client requests a TCP connection.
4	0.006499	Client to Server	Handshake step 2: [SYN, ACK] - Server acknowledges the request and agrees to connect.
5	0.006899	Client to Server	Handshake step 3: [ACK] - Client acknowledges the server, completing the handshake.
6	0.008699	Client to Server	First HTTP request: GET / HTTP/1.1 - Client requests the main HTML webpage.
8	0.070299	Client to Server	Server response to it: HTTP/1.1 200 OK - Server successfully responds with the HTML webpage.
14	0.184500	Client to Server	Connection closing, step 1: [FIN, ACK] - Client initiates the TCP connection shutdown.
15	0.185000	Client to Server	Connection closing, step 2: [FIN, ACK] - Server acknowledges and initiates its own shutdown.
16	0.185500	Client to Server	Connection closing, step 3: [ACK] - Client acknowledges the server's FIN, completing the normal closing sequence.

2D. Bonus: the reset

Question	Your answer
Frame number of the RST packet, who sent it, and to which port	Frame 18. The server (10.0.0.5) sent the [RST, ACK] packet to the workstation (10.0.0.22) on destination port 50395.

Question	Your answer
Which earlier packet triggered it? Why did the server reset instead of answering with SYN/ACK?	The exact triggering packet cannot be confirmed from this screenshot alone. The server may have reset the connection because the client sent a packet to a closed or unavailable service, or because the connection was no longer valid. A SYN/ACK would be expected in response to a valid connection request, not necessarily to a packet arriving on an invalid or closed connection.
Filter you could use to find every reset in a large capture	tcp.flags.reset == 1

Checkpoint: *Analyze, Expert Information* should list exactly one **Warning**, a connection reset.

Break: 14:45 to 15:00

Save your files. Swap the driver and navigator roles.

Part 3: Filter scavenger hunt

15:00 to 15:40, 40 minutes, pairs, file: hunt1.pcapng

Goal: write display filters quickly and correctly. Open `hunt1.pcapng` (363 packets: a mix of DNS, web, TLS, resets, and errors). For each task, write your filter exactly as you typed it, and the **Displayed** count from the status bar.

#	Task	Your display filter	Count
1	All DNS traffic.	dns	20
2	DNS responses that say the name does not exist (NXDOMAIN).	dns.flags.response == 1 && dns.flags.rcode == 3	3
3	HTTP requests that use the POST method.	http.request.method == "POST"	2
4	TCP packets with the reset flag set.	tcp.flags.reset == 1	4
5	Everything to or from host 10.0.0.22, excluding DNS.	ip.addr == 10.0.0.22 && !dns	34
6	TLS Client Hello packets. Then add the Server Name as a column and list the names below.	tls.handshake.type == 1	4
7	Packets longer than 1000 bytes that were sent by 10.0.0.23.	ip.src == 10.0.0.23 && frame.len > 1000	2
8	TCP retransmissions.	tcp.analysis.retransmission	3
9	ICMP destination unreachable messages.	icmp.type == 3	2
10	TCP traffic on any port other than 53, 80, and 443.	tcp && !(tcp.port == 53 || tcp.port == 80 || tcp.port == 443)	34

Server names from task 6: _www.example.com, docs.example.com, news.example.org, cdn.example.net_

Self-check

When you have finished, add up your ten counts. The total should be **108**. If yours is different, re-check the rows that look most suspicious. (Wrong filters often return 0 or the whole capture.)

Bonus

- **B1.** Open *Analyze, Expert Information*. How many **Connection reset (RST)** warnings are listed? _4_
- **B2.** Write **one** filter that shows both the NXDOMAIN responses and the TCP resets together (use `||`). Filter: _(dns.flags.response == 1 && dns.flags.rcode == 3) || (tcp.flags.reset == 1)_ Count: _7_

Field hints (use only if stuck)

Fields worth knowing: `dns.flags.rcode`, `http.request.method`, `tcp.flags.reset`, `tcp.analysis.retransmission`, `icmp.type`, `tls.handshake.type`, `frame.len`, `ip.src`, `ip.addr`, `tcp.port`.

You can build any filter by clicking: right-click a field in the middle pane, **Apply as Filter, Selected**. To combine, use **Prepare as Filter** and add `&&` (and), `||` (or), `!` (not).

For task 10, use `==` with `||` inside brackets, then negate with `!`.

Part 4: Baseline dossier

15:40 to 16:20, 40 minutes, pairs, file: office_day.pcapng

Goal: describe "normal" for this network using Wireshark's Statistics tools. You will compare against your dossier in Part 5, so be accurate. Open `office_day.pcapng`, a 30-minute capture of an ordinary morning for four workstations (10.0.0.21 to 10.0.0.24).

4A. Capture facts (Statistics, Capture File Properties)

Item	Value
Packets	4285
Elapsed time (minutes and seconds)	29 minutes & 50 seconds
Average packet size (bytes)	701b

4B. Protocol mix (Statistics, Protocol Hierarchy)

Read the **Percent Packets** column. Each protocol has its own row; expand the tree if you cannot see it.

Protocol	% of packets	Protocol	% of packets
TCP	90.8%	HTTP	1.6%
UDP	7.5%	DNS	3.6%
ARP	1.5%	Multicast DNS (mDNS)	2.8%
ICMP	0.2%	NTP	1.1%
TLS	10.3%		

4C. Busiest internal hosts (Statistics, Endpoints, IPv4 tab; click the Bytes column to sort)

Rank	IP address	Total bytes	Notes
1	10.0.0.21	2 MB	By far the busiest host
2	10.0.0.23	413 kB	Second-highest byte count
3	10.0.0.22	410 kB	Very close to rank 2

Only count internal hosts (addresses starting with 10.0.0.). Which host is far ahead of the others? What do you think it was doing (see 4F)?

4D. External destinations

On the same Endpoints (IPv4) tab, count the addresses that are **not** internal (not 10.0.0.x) and not multicast (224.x.x.x). Number of external IPs: _9_. Write them here:

203.0.113.123, 203.0.113.50, 198.51.100.40, 198.51.100.25, 198.51.100.60, 203.0.113.20, 203.0.113.10, 203.0.113.30, 198.51.100.90

4E. DNS activity

Item	Value
Number of DNS queries: apply `dns.flags.response == 0 && udp.port == 53`	77
Queries per minute (queries divided by minutes)	2.57 queries/minute (77 ÷ 30 minutes)
Why do we add `udp.port == 53` instead of using only `dns.flags.response == 0`?	It restricts the results to DNS traffic using UDP port 53, excluding DNS queries carried over other transports.

4F. Largest conversation (Statistics, Conversations, TCP tab; click Bytes)

Item	Value
Address A and Address B of the largest conversation	Address A: 10.0.0.21 Address B: 198.51.100.60
Total bytes and duration	24 kB & 0.222202 seconds (approximately 0.222 seconds)
The server's name: apply `ip.addr == <server IP> && tls.handshake.type == 1`, open the packet, expand *Handshake Protocol: Client Hello, Extension: server_name*	search.example.net

4G. Things that happen on a schedule (these are normal!)

For each row, apply the filter, then add the column **frame.time_delta_displayed** (expand the *Frame* section in the middle pane, right-click *Time delta from previous displayed frame*, **Apply as Column**). Read the gaps between packets.

Traffic	Filter	Typical gap (seconds)
Time synchronization	`ntp && ip.src == 10.0.0.21`	Approximately 300 seconds (5 minutes)
Operating system update check	`tls.handshake.extensions_server_name == "update.example.net" && ip.src == 10.0.0.21`	Approximately 600 seconds (10 minutes)
Local network chatter (mDNS)	`mdns && ip.src == 10.0.0.21`	between _47.4_ and _74.2_ seconds

Why is it important to know these regular patterns before you start hunting for threats?

Knowing the network's normal patterns helps distinguish routine activity, such as scheduled time synchronization, update checks, and mDNS traffic, from unusual behavior. Without a baseline, legitimate activity could be mistaken for a threat, while genuinely suspicious changes could go unnoticed.

4H. One-sentence baseline

Complete: "A normal 30 minutes on this network looks like..."

A normal 30 minutes on this network looks like routine TCP and UDP communications, DNS queries, ARP and mDNS local network activity, time synchronization from workstation 10.0.0.21 approximately every 5 minutes, and TLS connections to external servers, including periodic update checks approximately every 10 minutes.

Part 5: Odd one out (team challenge)

16:20 to 16:55, 35 minutes, teams of 3 to 4, file: tuesday.pcapng

Scenario. It is the next morning. The SOC captured the same 30 minutes on Tuesday. **Exactly four things in this capture do not belong.** Find them, prove each one with evidence, and rate how serious it is. The team with the best evidence wins.

Steps

1. Set **View, Time Display Format, UTC Date and Time of Day**. Open `tuesday.pcapng` (7,019 packets).
2. Compare against your Part 4 dossier: protocol mix, busiest conversations, DNS, ARP. What has changed?
3. For each anomaly, record **who** (IP addresses), **when** (UTC), **evidence** (the exact filter and frame numbers, and take a screenshot), **severity** (Low, Medium, or High, with a reason), and an **ATT&CK technique** from the list below.
4. Note anything that looked odd but turned out to be normal, and explain why (use your baseline as proof). A false alarm with no evidence **loses a point**.

Findings

#	What happened (who, to what)	When (UTC)	Evidence: filter and frame numbers	Severity and why	ATT&CK
1	Workstation 10.0.0.23 connected to FTP server 10.0.0.6. The FTP session transmitted the username and password through plaintext FTP commands, and the server subsequently confirmed a successful login.	September 15, 2026, 10:07:12.812300–10:07:14.045000 UTC for the username submission, password submission, and successful login sequence.	**Filter:** ftp || ftp-data **Frames:** 787, 791, 793 **Screenshot/s:** See Figure 1 below	High. Credentials transmitted in plaintext can be intercepted by someone with access to the network traffic.	T1552, Unsecured Credentials.
2	Workstation 10.0.0.21 sent an HTTP POST request to 198.51.100.77 on port 8080, uploading a file named backup.zip.	September 15, 2026, 10:24:20.624543 UTC.	**Filter:** tcp.stream eq 82 **Frames:** 6528, 6531 **Screenshot/s:** See Figure 2 below	High, suspected data exfiltration. The upload is confirmed, but whether it was authorized or involved sensitive data has not been established.	T1048, Exfiltration Over Alternative Protocol, if the transfer is confirmed to be unauthorized exfiltration.

#	What happened (who, to what)	When (UTC)	Evidence: filter and frame numbers	Severity and why	ATT&CK
3	Workstation 10.0.0.24 sent a rapid sequence of ARP requests asking for the MAC addresses associated with multiple internal IP addresses. The displayed sequence includes targets from 10.0.0.3 through 10.0.0.18.	September 15, 2026, approximately 10:12:30.050000–10:12:30.360000 UTC for the displayed sequence.	**Filter:** arp.src.proto_ipv4 == 10.0.0.24 **Frames:** 2017–2036 as the inspected range **Screenshot/s:** See Figure 3 below	Medium. Rapid requests targeting multiple internal IP addresses are consistent with host discovery, although ARP traffic can also occur during normal network activity.	T1018, Remote System Discovery.
4	Workstation 10.0.0.22 generated numerous DNS queries for random-looking subdomains under example.net. DNS server 10.0.0.2 returned repeated NXDOMAIN responses, indicating that the requested names did not exist.	September 15, 2026, approximately 10:18:05–10:18:51 UTC.	**Query filter:** dns.flags.response == 0 && ip.src == 10.0.0.22 **Response filter:** dns.flags.response == 1 && dns.flags.rcode == 3 && ip.src == 10.0.0.2 **Screenshot/s:** See Figure 4.1 and 4.2 below	High, suspected malicious DNS behavior, pending further validation.	T1568.002, Dynamic Resolution: Domain Generation Algorithms.

FIGURES

Anomaly Screenshots can also be accessed here:
https://drive.google.com/drive/folders/1f6cGEMaFEVwVXjQ-onPpacgJLftyea1E?usp=sharing

Figure 1

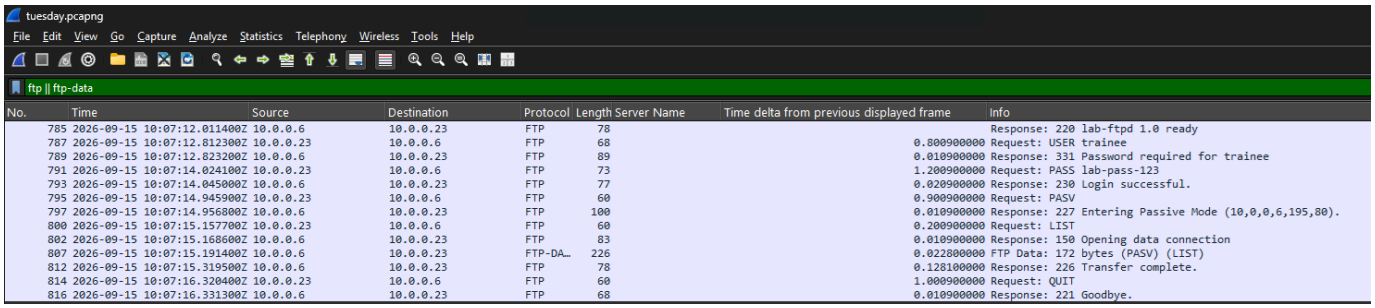


Figure 2

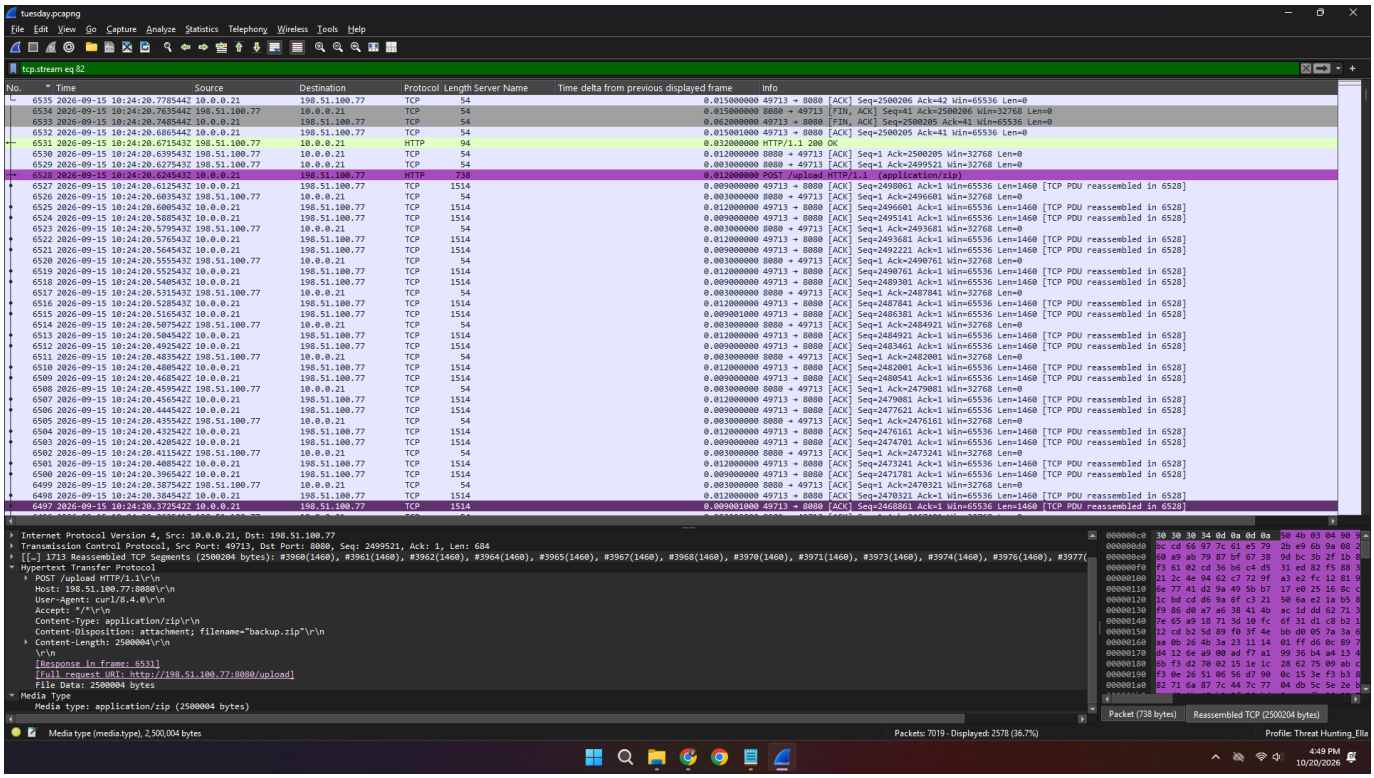


Figure 3

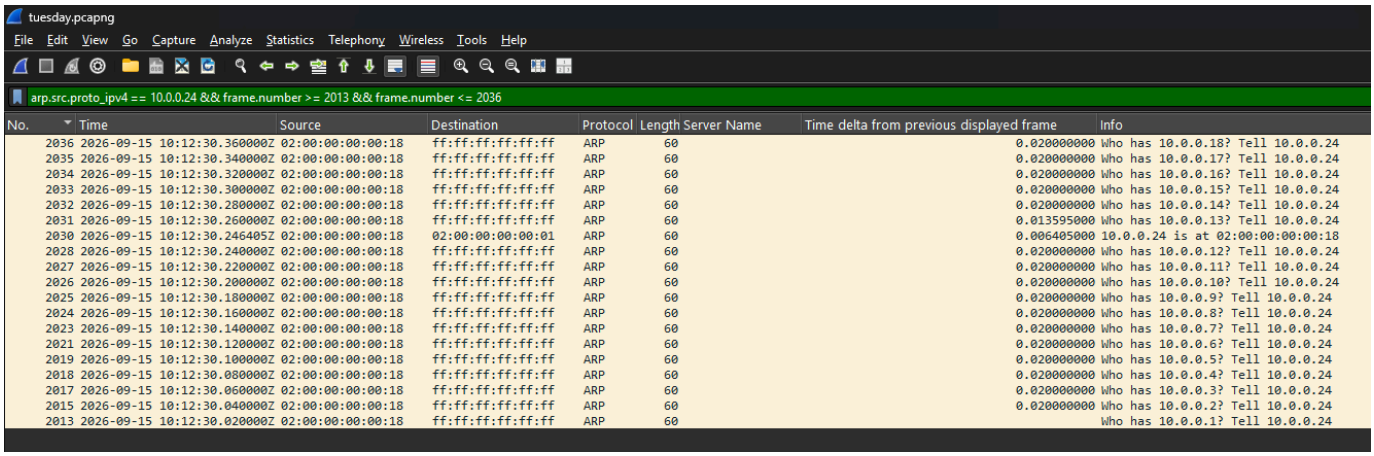


Figure 4.1

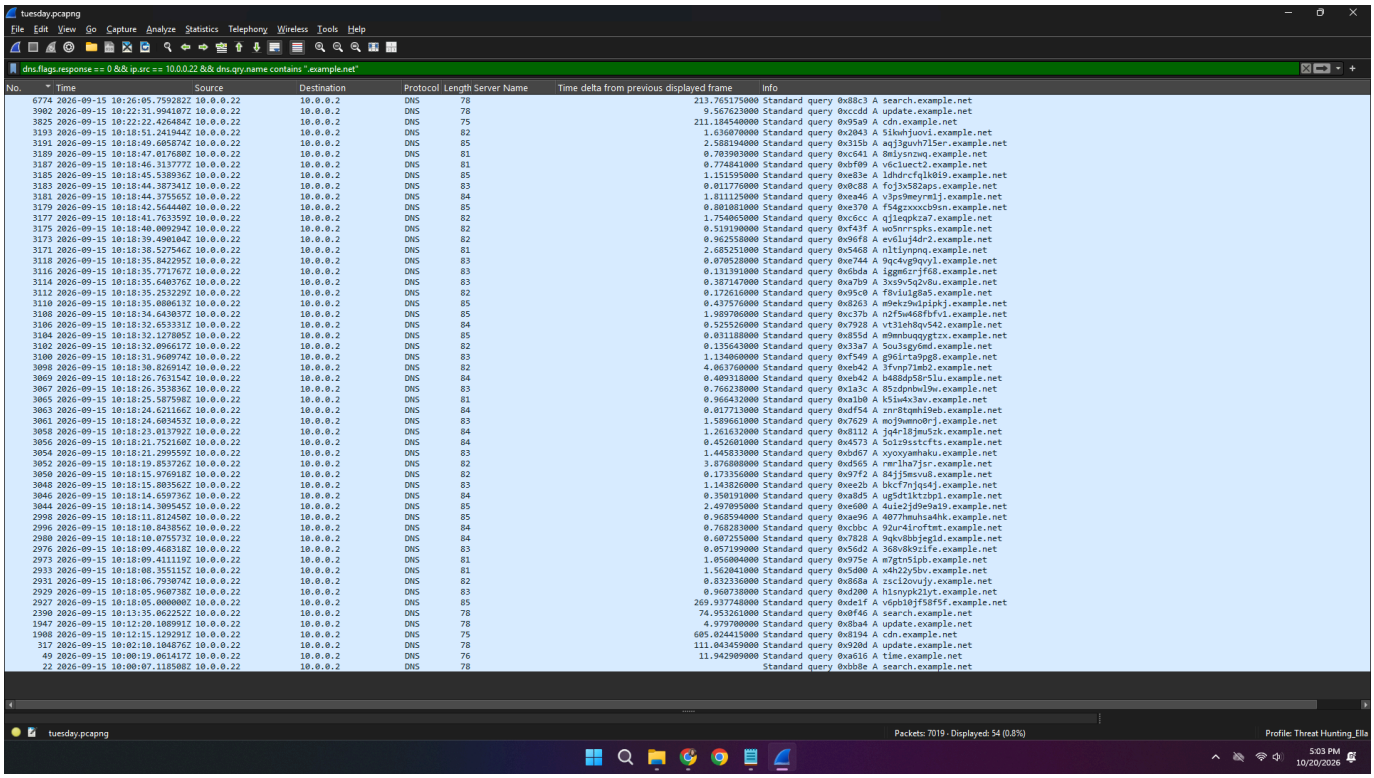
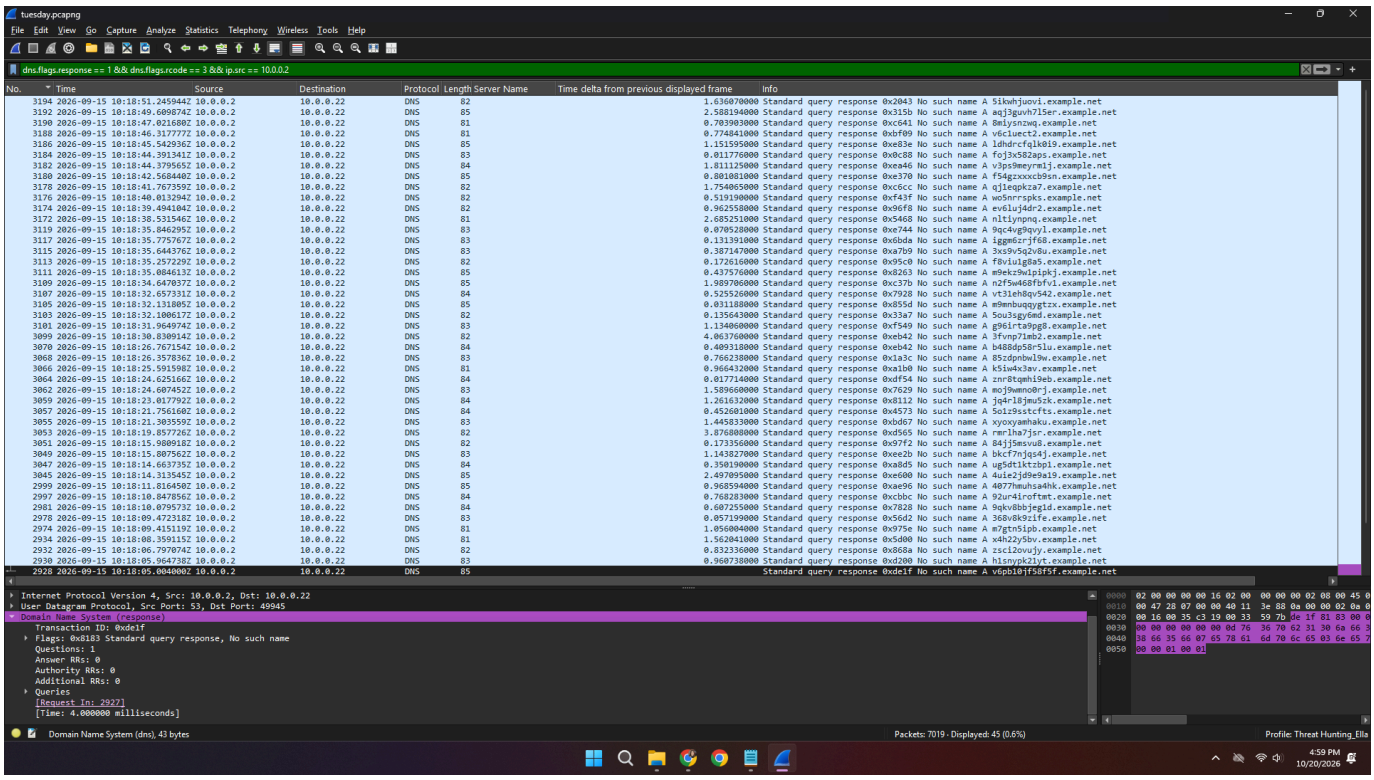


Figure 4.2



Looked odd, but is normal

What looked odd	Why it is normal (evidence from your baseline)
Periodic NTP requests from 10.0.0.21 to 203.0.113.123	Filter: ntp && ip.src == 10.0.0.21. Frames 65, 501, 1728, 2181, 2783, and 3860 show intervals of approximately 299–301 seconds.
Recurring mDNS queries from 10.0.0.21 to 224.0.0.251	Filter: mdns && ip.src == 10.0.0.21. The capture shows recurring PTR queries for _services._dns-sd._udp.local at intervals of approximately 47–74 seconds.

ATT&CK techniques to choose from

ID	Technique
T1018	Remote System Discovery (finding other hosts on the network)
T1046	Network Service Discovery (finding open ports and services)
T1040	Network Sniffing (credentials or data exposed to anyone on the path)
T1552	Unsecured Credentials (passwords stored or sent in a readable form)
T1568.002	Dynamic Resolution: Domain Generation Algorithms
T1071.004	Application Layer Protocol: DNS
T1041	Exfiltration Over C2 Channel
T1048	Exfiltration Over Alternative Protocol
T1560	Archive Collected Data

Hint cards (open only after 15 minutes of searching)

Hint 1. Compare Protocol Hierarchy with your Part 4B table. Is there a protocol that was not there yesterday?

Hint 2. Statistics, Conversations, TCP tab, sort by Bytes. Compare the top row with your Part 4F answer.

Hint 3. Statistics, Endpoints, **Ethernet** tab, sort by Packets. Then filter `arp` and read the Info column.

Hint 4. Filter `dns.flags.rcode == 3` and read the names in the Info column. Do they look like real words?

Submit and self-check

16:55 to 17:00

- ☑ I saved `Part1_yourinitials.pcapng` (or used the fallback file).
- ☑ Every table in this worksheet is filled in (or marked "not done" with a reason).
- ☑ My team's Part 5 findings include a filter, frame numbers, and a screenshot for each anomaly.
- ☑ Names of both partners are at the top of page 1.

Can I do this now? (tick what you can do without help)

- ☑ Start and stop a capture and save it as pcapng.
- ☑ Explain the difference between a capture filter and a display filter.
- ☑ Read a TCP handshake and a DNS query and response in Packet Details.
- ☑ Write display filters for DNS errors, resets, retransmissions, and ICMP errors.
- ☑ Build a baseline with Protocol Hierarchy, Conversations, and Endpoints.
- ☑ Explain why baselines matter before hunting for threats.

One thing that clicked today: _How to identify suspicious network activity by correlating packet timestamps, source and destination IPs, and protocol behavior, rather than relying on a single unusual packet._

One thing that is still fuzzy: _How to distinguish genuinely malicious DNS activity, such as domain generation algorithm (DGA) behavior, from legitimate traffic that happens to look suspicious._

Appendix A: Quick reference

I want to...	Do this
Open a capture file	File, Open (`Ctrl+O`)
Start or stop capturing	`Ctrl+E`
Change capture options	`Ctrl+K`
Jump to the filter bar	`Ctrl+/`
Clear the filter	Click the X at the right end of the filter bar
Turn any field into a filter	Right-click the field, Apply as Filter, Selected
Add a field as a column	Right-click the field, Apply as Column
Read a whole conversation	Right-click a packet, Follow, TCP Stream (`Ctrl+Alt+Shift+T`)
See warnings and errors quickly	Analyze, Expert Information
Capture facts (duration, average size)	Statistics, Capture File Properties
Protocol percentages	Statistics, Protocol Hierarchy
Top talkers	Statistics, Endpoints (each host) or Conversations (each pair)
Traffic over time	Statistics, I/O Graphs
Change how time is shown	View, Time Display Format

Filter operators

Operator	Meaning	Example
`==` and `!=`	equals, does not equal	`ip.addr == 10.0.0.5`
`>` `<` `>=` `<=`	greater, less	`frame.len > 1000`
`&&`	and (both must be true)	`dns && ip.src == 10.0.0.22`
`||`	or (either can be true)	`arp || icmp`
`!`	not	`!dns`
`contains`	field includes this text	`http.host contains "lab"`

Capture filter examples (used **before** capturing): `host 10.0.0.5`, `port 53`, `not port 22`, `net 10.0.0.0/24`.

Appendix B: Troubleshooting

Problem	Fix
The filter bar is red	There is a syntax error. Check spelling and that text values are in double quotes, for example `http.host == "example.com"`. Use `==` (two equals signs), not `=`.
The filter bar is yellow	The filter works but may be ambiguous. Usually safe; check the count.

Problem	Fix
I applied a filter and see 0 packets	The filter is valid but nothing matches. Re-check the field name, or clear the filter and look at a real packet to copy the field.
I cannot see any interfaces, or capture fails	Wireshark lacks capture permission or the capture driver is missing. Tell the instructor and use the fallback file for Part 1.
The Time column shows dates instead of seconds	View, Time Display Format, choose *Seconds Since Beginning of Capture* (Parts 2 to 4) or *UTC Date and Time of Day* (Part 5).
I see host names instead of IP addresses	Name resolution is on. View, Name Resolution, untick all three items.
`in {80 443}` gives an error	In Wireshark 4.x the list must use commas: `tcp.port in {80, 443}`. Or use `||` as in Part 3.
My counts differ slightly from a classmate's	Check the Status bar shows Displayed, not Packets. Displayed is the filtered count.
Wireshark froze on a large file	Wait a moment (the 5 MB file needs a few seconds). If it hangs, restart and re-open the file.